ML Classification Final Project

April 2025

Objective

The purpose of this project is to identify which classification method performs best in predicting whether a customer will stay or cancel their service.

To do this, metrics such as precision, recall, f1score, accuracy, etc. will be calculated for each model, using the same training-test dataset.

Content

Install and setup libraries

In [299…

```
!pip install pandas
!pip install numpy
!pip install matplotlib
!pip install seaborn
!pip install scikit-learn
!pip install xgboost
!pip install nbconvert
```

```
Requirement already satisfied: pandas in ./.venv/lib/python3.10/site-packages (2.2.3)
Requirement already satisfied: numpy>=1.22.4 in ./.venv/lib/python3.10/site-packages (from pandas) (2.2.4)
Requirement already satisfied: python-dateutil>=2.8.2 in ./.venv/lib/python3.10/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in ./.venv/lib/python3.10/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in ./.venv/lib/python3.10/site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in ./.venv/lib/python3.10/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: numpy in ./.venv/lib/python3.10/site-packages (2.2.4)
Requirement already satisfied: matplotlib in ./.venv/lib/python3.10/site-packages (3.10.1)
Requirement already satisfied: contourpy>=1.0.1 in ./.venv/lib/python3.10/site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in ./.venv/lib/python3.10/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./.venv/lib/python3.10/site-packages (from matplotlib) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in ./.venv/lib/python3.10/site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in ./.venv/lib/python3.10/site-packages (from matplotlib) (2.2.4)
Requirement already satisfied: packaging>=20.0 in ./.venv/lib/python3.10/site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in ./.venv/lib/python3.10/site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in ./.venv/lib/python3.10/site-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in ./.venv/lib/python3.10/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in ./.venv/lib/python3.10/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: seaborn in ./.venv/lib/python3.10/site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in ./.venv/lib/python3.10/site-packages (from seaborn) (2.2.4)
Requirement already satisfied: pandas>=1.2 in ./.venv/lib/python3.10/site-packages (from seaborn) (2.2.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in ./.venv/lib/python3.10/site-packages (from seaborn) (3.10.1)
Requirement already satisfied: contourpy>=1.0.1 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in ./.venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in ./.venv/lib/python3.10/site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in ./.venv/lib/python3.10/site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in ./.venv/lib/python3.10/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Requirement already satisfied: scikit-learn in ./.venv/lib/python3.10/site-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in ./.venv/lib/python3.10/site-packages (from scikit-learn) (2.2.4)
Requirement already satisfied: scipy>=1.6.0 in ./.venv/lib/python3.10/site-packages (from scikit-learn) (1.15.2)
Requirement already satisfied: joblib>=1.2.0 in ./.venv/lib/python3.10/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in ./.venv/lib/python3.10/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: xgboost in ./.venv/lib/python3.10/site-packages (3.0.0)
Requirement already satisfied: numpy in ./.venv/lib/python3.10/site-packages (from xgboost) (2.2.4)
Requirement already satisfied: nvidia-nccl-cu12 in ./.venv/lib/python3.10/site-packages (from xgboost) (2.26.2.post1)
Requirement already satisfied: scipy in ./.venv/lib/python3.10/site-packages (from xgboost) (1.15.2)
Requirement already satisfied: nbconvert in ./.venv/lib/python3.10/site-packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in ./.venv/lib/python3.10/site-packages (from nbconvert) (4.13.3)
Requirement already satisfied: bleach!=5.0.0 in ./.venv/lib/python3.10/site-packages (from bleach[css]!=5.0.0->nbconvert) (6.2.0)
Requirement already satisfied: defusedxml in ./.venv/lib/python3.10/site-packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in ./.venv/lib/python3.10/site-packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in ./.venv/lib/python3.10/site-packages (from nbconvert) (5.7.2)
Requirement already satisfied: jupyterlab-pygments in ./.venv/lib/python3.10/site-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in ./.venv/lib/python3.10/site-packages (from nbconvert) (3.0.2)
Requirement already satisfied: mistune<4,>=2.0.3 in ./.venv/lib/python3.10/site-packages (from nbconvert) (3.1.3)
Requirement already satisfied: nbclient>=0.5.0 in ./.venv/lib/python3.10/site-packages (from nbconvert) (0.10.2)
Requirement already satisfied: nbformat>=5.7 in ./.venv/lib/python3.10/site-packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in ./.venv/lib/python3.10/site-packages (from nbconvert) (24.2)
Requirement already satisfied: pandocfilters>=1.4.1 in ./.venv/lib/python3.10/site-packages (from nbconvert) (1.5.1)
Requirement already satisfied: pygments>=2.4.1 in ./.venv/lib/python3.10/site-packages (from nbconvert) (2.19.1)
Requirement already satisfied: traitlets>=5.1 in ./.venv/lib/python3.10/site-packages (from nbconvert) (5.14.3)
Requirement already satisfied: webencodings in ./.venv/lib/python3.10/site-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in ./.venv/lib/python3.10/site-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)
Requirement already satisfied: platformdirs>=2.5 in ./.venv/lib/python3.10/site-packages (from jupyter-core>=4.7->nbconvert) (4.3.7)
Requirement already satisfied: typing-extensions in ./.venv/lib/python3.10/site-packages (from mistune<4,>=2.0.3->nbconvert) (4.13.2)
Requirement already satisfied: jupyter-client>=6.1.12 in ./.venv/lib/python3.10/site-packages (from nbclient>=0.5.0->nbconvert) (8.6.3)
Requirement already satisfied: fastjsonschema>=2.15 in ./.venv/lib/python3.10/site-packages (from nbformat>=5.7->nbconvert) (2.21.1)
Requirement already satisfied: jsonschema>=2.6 in ./.venv/lib/python3.10/site-packages (from nbformat>=5.7->nbconvert) (4.23.0)
Requirement already satisfied: soupsieve>1.2 in ./.venv/lib/python3.10/site-packages (from beautifulsoup4->nbconvert) (2.6)
Requirement already satisfied: attrs>=22.2.0 in ./.venv/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in ./.venv/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (2024.10.1)
Requirement already satisfied: referencing>=0.28.4 in ./.venv/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.36.2)
Requirement already satisfied: rpds-py>=0.7.1 in ./.venv/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.24.0)
Requirement already satisfied: python-dateutil>=2.8.2 in ./.venv/lib/python3.10/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in ./.venv/lib/python3.10/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.4.0)
Requirement already satisfied: tornado>=6.2 in ./.venv/lib/python3.10/site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.4.2)
Requirement already satisfied: six>=1.5 in ./.venv/lib/python3.10/site-packages (from python-dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
```

In [2]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LogisticRegressionCV

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import f1_score
```

```
from sklearn.metrics import roc_auc_score
from sklearn.metrics import precision_recall_fscore_support
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import roc_curve
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

from sklearn.preprocessing import label_binarize
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import VotingClassifier
from xgboost import XGBClassifier
```

Obtaining dataset

In [3]:
```
data = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-ML241EN-SkillsNetwork/labs/datasets/churndata_processed.csv")
data.head()
```

Out[3]:

	months	multiple	gb_mon	security	backup	protection	support	unlimited	contract	paperless	...	payment_Credit Card	payment_Mailed Check	internet_type_DSL	internet_type_Fiber Optic	internet_type_None	offer_Offer A	of
0	0.00	0	0.094118	0	0	1	0	0	0.0	1	...	0	0	1	0	0	0	
1	0.00	1	0.200000	0	1	0	0	1	0.0	1	...	1	0	0	1	0	0	
2	0.25	1	0.611765	0	0	0	0	1	0.0	1	...	0	0	0	1	0	0	
3	0.25	0	0.141176	0	1	1	0	1	0.0	1	...	0	0	0	1	0	0	
4	0.50	1	0.164706	0	0	0	0	1	0.0	1	...	0	0	0	1	0	0	

5 rows × 23 columns

Analize dataset

In [129…
```
data.shape
```

Out[129…
```
(7043, 23)
```

In [4]:
```
data.dtypes
```

Out[4]:
```
months                          float64
multiple                          int64
gb_mon                          float64
security                          int64
backup                            int64
protection                        int64
support                           int64
unlimited                         int64
contract                        float64
paperless                         int64
monthly                         float64
satisfaction                    float64
churn_value                       int64
payment_Credit Card               int64
payment_Mailed Check              int64
internet_type_DSL                 int64
internet_type_Fiber Optic         int64
internet_type_None                int64
offer_Offer A                     int64
offer_Offer B                     int64
offer_Offer C                     int64
offer_Offer D                     int64
offer_Offer E                     int64
dtype: object
```

In [5]:
```
data.isnull().sum()
```

Out[5]:
```
months                          0
multiple                        0
gb_mon                          0
security                        0
backup                          0
protection                      0
support                         0
unlimited                       0
contract                        0
paperless                       0
monthly                         0
satisfaction                    0
churn_value                     0
payment_Credit Card             0
payment_Mailed Check            0
internet_type_DSL               0
internet_type_Fiber Optic       0
internet_type_None              0
offer_Offer A                   0
offer_Offer B                   0
offer_Offer C                   0
offer_Offer D                   0
offer_Offer E                   0
dtype: int64
```

In [6]:
```
data.describe().T
```

Out[6]:

	count	mean	std	min	25%	50%	75%	max
months	7043.0	0.433551	0.398231	0.0	0.000000	0.250000	0.750000	1.0
multiple	7043.0	0.421837	0.493888	0.0	0.000000	0.000000	1.000000	1.0
gb_mon	7043.0	0.241358	0.240223	0.0	0.035294	0.200000	0.317647	1.0
security	7043.0	0.286668	0.452237	0.0	0.000000	0.000000	1.000000	1.0
backup	7043.0	0.344881	0.475363	0.0	0.000000	0.000000	1.000000	1.0
protection	7043.0	0.343888	0.475038	0.0	0.000000	0.000000	1.000000	1.0
support	7043.0	0.290217	0.453895	0.0	0.000000	0.000000	1.000000	1.0
unlimited	7043.0	0.673719	0.468885	0.0	0.000000	1.000000	1.000000	1.0
contract	7043.0	0.377396	0.424234	0.0	0.000000	0.000000	1.000000	1.0
paperless	7043.0	0.592219	0.491457	0.0	0.000000	1.000000	1.000000	1.0
monthly	7043.0	0.462803	0.299403	0.0	0.171642	0.518408	0.712438	1.0
satisfaction	7043.0	0.561231	0.300414	0.0	0.500000	0.500000	0.750000	1.0
churn_value	7043.0	0.265370	0.441561	0.0	0.000000	0.000000	1.000000	1.0
payment_Credit Card	7043.0	0.390317	0.487856	0.0	0.000000	0.000000	1.000000	1.0
payment_Mailed Check	7043.0	0.054664	0.227340	0.0	0.000000	0.000000	0.000000	1.0
internet_type_DSL	7043.0	0.234559	0.423753	0.0	0.000000	0.000000	0.000000	1.0
internet_type_Fiber Optic	7043.0	0.430924	0.495241	0.0	0.000000	0.000000	1.000000	1.0
internet_type_None	7043.0	0.216669	0.412004	0.0	0.000000	0.000000	0.000000	1.0
offer_Offer A	7043.0	0.073832	0.261516	0.0	0.000000	0.000000	0.000000	1.0
offer_Offer B	7043.0	0.116996	0.321438	0.0	0.000000	0.000000	0.000000	1.0
offer_Offer C	7043.0	0.058924	0.235499	0.0	0.000000	0.000000	0.000000	1.0
offer_Offer D	7043.0	0.085475	0.279607	0.0	0.000000	0.000000	0.000000	1.0
offer_Offer E	7043.0	0.114298	0.318195	0.0	0.000000	0.000000	0.000000	1.0

In [7]:
```
data_uniques = pd.DataFrame([[i, len(data[i].unique())] for i in data.columns],columns=['Variable', 'Unique Values']).set_index('Variable')
data_uniques
```

Out[7]:

Variable	Unique Values
months	5
multiple	2
gb_mon	50
security	2
backup	2
protection	2
support	2
unlimited	2
contract	3
paperless	2
monthly	1585
satisfaction	5
churn_value	2
payment_Credit Card	2
payment_Mailed Check	2
internet_type_DSL	2
internet_type_Fiber Optic	2
internet_type_None	2
offer_Offer A	2
offer_Offer B	2
offer_Offer C	2
offer_Offer D	2
offer_Offer E	2

Defining Features and Target dataset

In [8]:
```
features = data.drop('churn_value', axis=1)
target = data['churn_value']
```

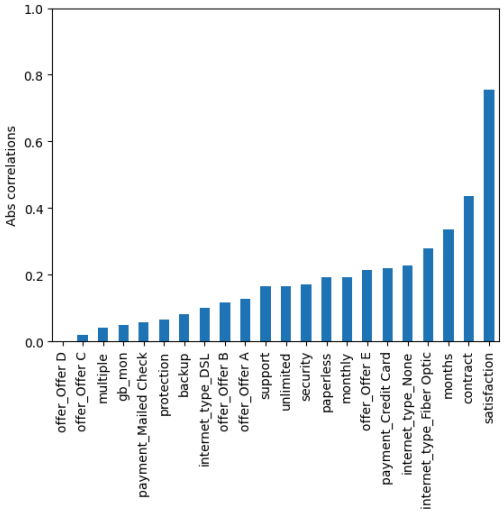
Identifying correlation between features and target dataset

In [9]:
```
correlations = features.corrwith(target)
correlations.sort_values(inplace=True)
correlations
```

Out[9]:
```
satisfaction               -0.754649
contract                   -0.435398
months                     -0.337205
internet_type_None         -0.227890
payment_Credit Card        -0.218528
security                   -0.171226
support                    -0.164674
offer_Offer A              -0.126654
offer_Offer B              -0.117723
internet_type_DSL          -0.099716
backup                     -0.082255
protection                 -0.066160
offer_Offer C              -0.020660
offer_Offer D               0.001435
multiple                    0.040102
gb_mon                      0.048868
payment_Mailed Check        0.056348
unlimited                   0.166545
paperless                   0.191825
monthly                     0.193356
offer_Offer E               0.214648
internet_type_Fiber Optic   0.279623
dtype: float64
```

In [10]:
```
ax = correlations.abs().sort_values().plot(kind='bar')
ax.set(ylim=[0, 1], ylabel='Abs correlations')
```

Out[10]:
```
[(0.0, 1.0), Text(0, 0.5, 'Abs correlations')]
```

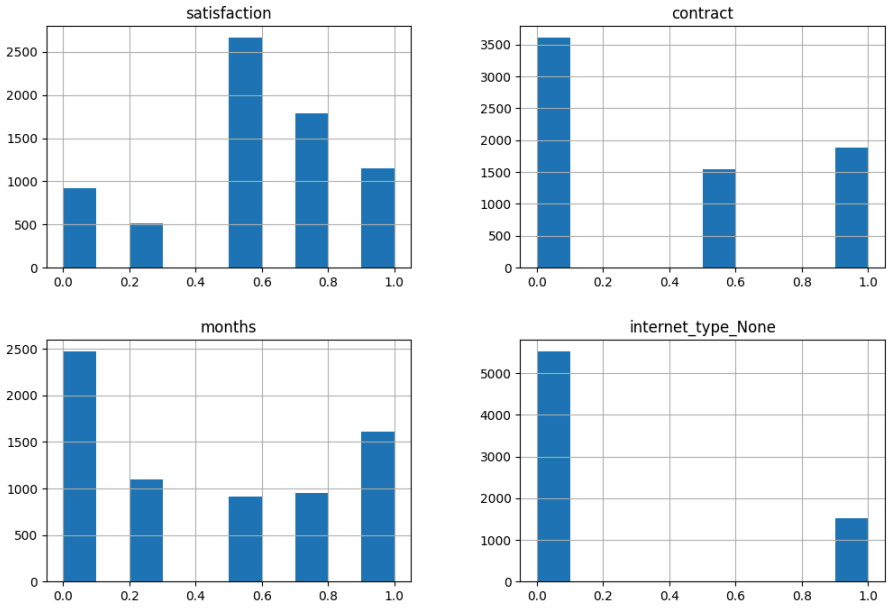


Identifying the distribution of the most relevant features

In [11]:
```
most_relevant_features = ['satisfaction','contract','months','internet_type_None']
```

In [12]:
```
data[most_relevant_features].hist(figsize=(12, 8))
```

Out[12]:
```
array([[<Axes: title={'center': 'satisfaction'}>,
        <Axes: title={'center': 'contract'}>],
       [<Axes: title={'center': 'months'}>,
        <Axes: title={'center': 'internet_type_None'}>]], dtype=object)
```



Creating training and test datasets

As seen in the graphs, the dataset is unbalanced so the training and test sets will be created in stratified mode.

In [13]:
```
X = features
y = target
```

In [14]:
```
X_train, X_test, y_train, y_test = train_test_split(features, target, test_size=0.3, stratify=y, random_state=42)
```

Logistic Regression

In [15]:
```
def warn(*args, **kwargs):
    pass
import warnings

warnings.warn = warn
```

In [61]:
```
def show_features_importance(model):
    feature_imp = pd.Series(model.feature_importances_, index=features.columns).sort_values(ascending=False)
    fig, ax = plt.subplots(figsize=(8, 6))
    ax.pie(feature_imp, labels=None, autopct=lambda pct: '{:1.1f}%'.format(pct)
    if pct > 5.5 else '')
    ax.set(ylabel='Relative Importance')
    ax.set(xlabel='Feature')

    # Adjust the layout to prevent label overlapping
    plt.tight_layout()

    # Move the legend outside the chart
    plt.legend(loc='center left', bbox_to_anchor=(1, 0.5),labels=feature_imp.index)
    plt.show()
```

In [16]:
```
def show_ROC_PressRecCurv(X_test, y_test, model, title):
    sns.set_context('talk')
    fig, axList = plt.subplots(ncols=2)
    fig.set_size_inches(12, 6)
    fig.suptitle(title)

    # Get the probabilities for each of the two categories
    y_prob = model.predict_proba(X_test)

    # Plot the ROC-AUC curve
    ax = axList[0]
```

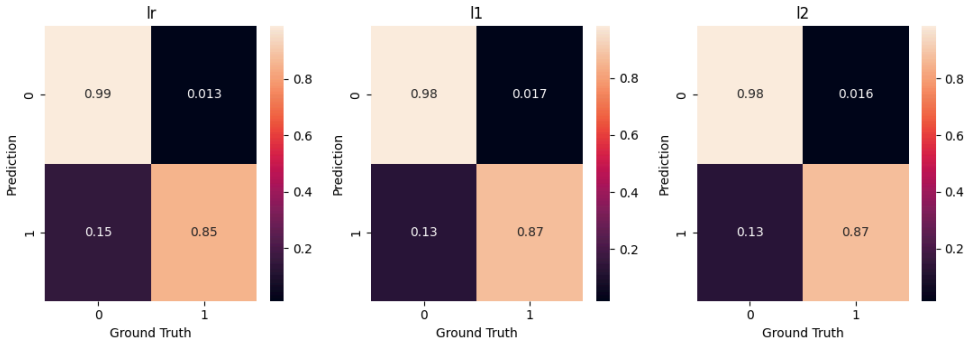
```
    fpr, tpr, thresholds = roc_curve(y_test, y_prob[:,1])
    ax.plot(fpr, tpr, linewidth=5)

    # It is customary to draw a diagonal dotted line in ROC plots.
    # This is to indicate completely random prediction. Deviation from this
    # dotted line towards the upper left corner signifies the power of the model.
    ax.plot([0, 1], [0, 1], ls='--', color='black', lw=.3)
    ax.set(xlabel='False Positive Rate', ylabel='True Positive Rate', xlim=[-.01, 1.01], ylim=[-.01, 1.01], title='ROC curve')
    ax.grid(True)

    # Plot the precision-recall curve
    ax = axList[1]
    precision, recall, _ = precision_recall_curve(y_test, y_prob[:,1])
    ax.plot(recall, precision, linewidth=5)
    ax.set(xlabel='Recall', ylabel='Precision',  xlim=[-.01, 1.01], ylim=[-.01, 1.01], title='Precision-Recall curve')
    ax.grid(True)
    plt.tight_layout()
```

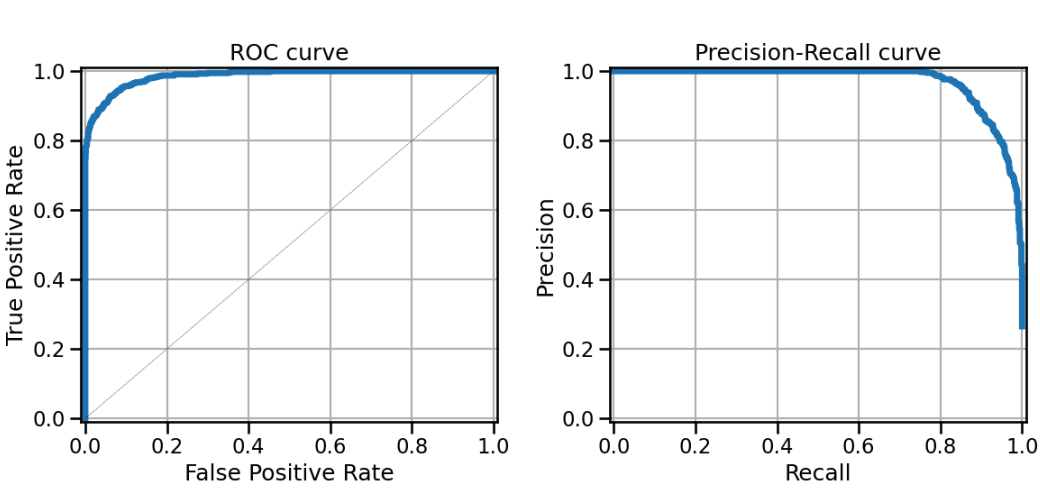
In [17]:
```
def show_heatmaps(param):
    coeff_labels = ['lr', 'l1', 'l2']
    fig, axList = plt.subplots(nrows=1, ncols=3)
    axList = axList.flatten()
    fig.set_size_inches(11, 4)
    #axList[-1].axis('off')
    for ax,lab in zip(axList, coeff_labels):
        sns.heatmap(param[lab], ax=ax, annot=True)
        ax.set(title=lab, ylabel='Prediction', xlabel='Ground Truth')

    plt.tight_layout()
```

In [18]:
```
def run_logisitc_regression(X_train, X_test, y_train, y_test):
    lr = LogisticRegression(solver='liblinear').fit(X_train, y_train)
    lr_l1 = LogisticRegressionCV(Cs=10, cv=4, penalty='l1',solver='liblinear').fit(X_train, y_train)
    lr_l2 = LogisticRegressionCV(Cs=10, cv=4, penalty='l2',solver='liblinear').fit(X_train, y_train)
    y_pred = list()
    y_prob = list()
    coeff_labels = ['lr', 'l1', 'l2']
    coeff_models = [lr, lr_l1, lr_l2]
    for lab, mod in zip(coeff_labels, coeff_models):
        y_pred.append(pd.Series(mod.predict(X_test), name=lab))
        y_prob.append(pd.Series(mod.predict_proba(X_test).max(axis=1), name=lab))

    y_pred = pd.concat(y_pred, axis=1)
    y_prob = pd.concat(y_prob, axis=1)

    metrics = list()
    cm = dict()
    for lab in coeff_labels:
        # Preciision, recall, f-score from the multi-class support function
        precision, recall, fscore, _ = precision_recall_fscore_support(y_test, y_pred[lab], average='weighted')
        # The usual way to calculate accuracy
        accuracy = accuracy_score(y_test, y_pred[lab])
        # ROC-AUC scores can be calculated by binarizing the data443
        auc = roc_auc_score(label_binarize(y_test, classes=[0,1,2,3,4,5]),label_binarize(y_pred[lab], classes=[0,1,2,3,4,5]),average='weighted')
        # Last, the confusion matrix
        cm[lab] = confusion_matrix(y_test, y_pred[lab], normalize='true')
        metrics.append(pd.Series({'precision':precision, 'recall':recall,'f1score':fscore, 'accuracy':accuracy,'auc':auc},name=lab))

    metrics = pd.concat(metrics, axis=1)
    print(metrics)
    show_heatmaps(cm)
    for lab, mod in zip(coeff_labels, coeff_models):
        show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=mod, title=lab)

    return metrics
```

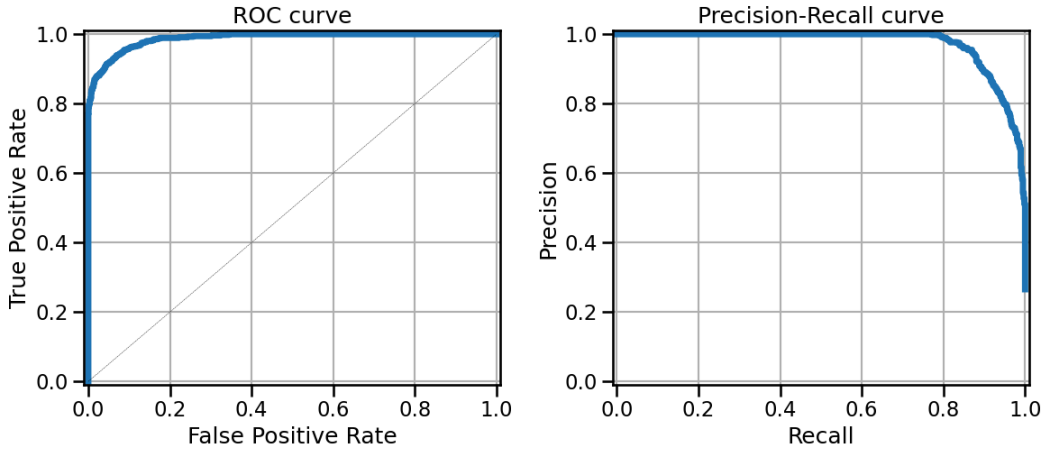
In [19]:
```
lr_metrics = run_logisitc_regression(X_train=X_train, X_test=X_test, y_train=y_train, y_test=y_test)
```

```
                lr        l1        l2
precision  0.950684  0.952993  0.954013
recall     0.950308  0.953147  0.954094
f1score    0.949289  0.952503  0.953433
accuracy   0.950308  0.953147  0.954094
auc        0.917799  0.927130  0.927775
```

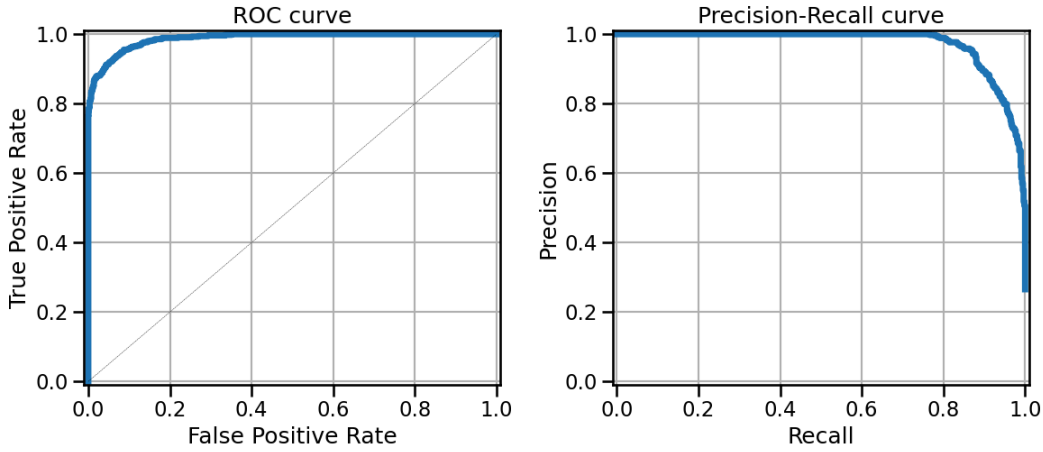




l1



l2



K-Nearest Neighbor

In [32]:
```
def evaluate_metrics(yt, yp):
    results_pos = {}
    results_pos['accuracy'] = round(accuracy_score(yt, yp), 3)
    precision, recall, f_beta, _ = precision_recall_fscore_support(yt, yp, average='weighted')
    results_pos['recall'] = round(recall, 3)
    results_pos['precision'] = round(precision, 3)
    results_pos['f1score'] = round(f_beta, 3)
    return results_pos
```

In [33]:
```
def kN_best_k(X_train, X_test, y_train, y_test):
    max_k = 40
    f1_scores = list()
    error_rates = list() # 1-accuracy
    for k in range(1, max_k):
        knn = KNeighborsClassifier(n_neighbors=k, weights='distance')
        knn = knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)
        f1 = f1_score(y_pred, y_test)
        f1_scores.append((k, round(f1_score(y_test, y_pred), 4)))
        error = 1-round(accuracy_score(y_test, y_pred), 4)
        error_rates.append((k, error))

    f1_results = pd.DataFrame(f1_scores, columns=['K', 'F1 Score'])
    error_results = pd.DataFrame(error_rates, columns=['K', 'Error Rate'])
    return f1_results, error_results
```

In [34]:
```
def knn_f1_score(f1_results):
    sns.set_context('talk')
    sns.set_style('ticks')
    #plt.figure(dpi=200)
    ax = f1_results.set_index('K').plot(figsize=(7, 6), linewidth=6)
    ax.set(xlabel='K', ylabel='F1 Score')
    ax.set_xticks(range(1, max_k, 2));
    plt.title('KNN F1 Score')
    plt.savefig('knn_f1.png')
```

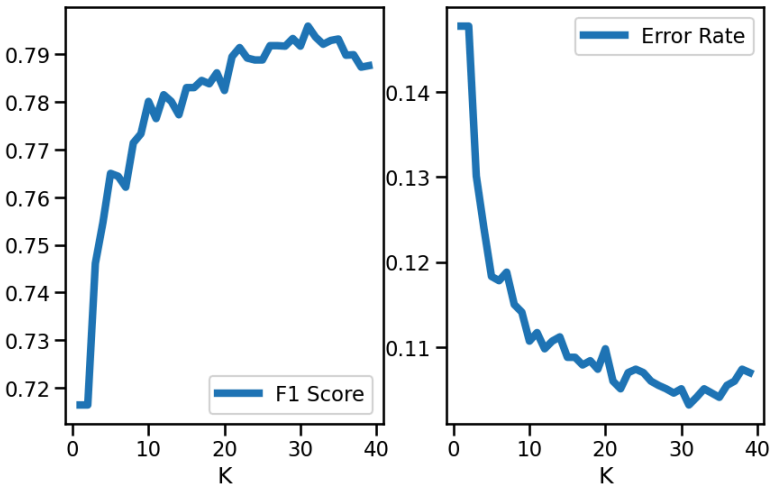
In [35]:
```
def knn_error(error_results):
    sns.set_context('talk')
    sns.set_style('ticks')
    plt.figure(dpi=300)
    ax = error_results.set_index('K').plot(figsize=(7, 6), linewidth=6)
    ax.set(xlabel='K', ylabel='Error Rate')
    ax.set_xticks(range(1, max_k, 2))
    plt.title('KNN Elbow Curve')
    plt.savefig('knn_elbow.png')
```

In [36]:
```
def knn_f1_error(f1_results, error_results):
    fig, (ax1, ax2) = plt.subplots(1,2)
    f1_results.set_index('K').plot(figsize=(10, 6), linewidth=6, ax=ax1)
    error_results.set_index('K').plot(figsize=(10, 6), linewidth=6, ax=ax2)
```

In [37]:
```
max_k = 40
f1_scores = list()
error_rates = list() # 1-accuracy
for k in range(1, max_k):
    knn = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn = knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    f1 = f1_score(y_pred, y_test)
```

```
    f1_scores.append((k, round(f1_score(y_test, y_pred), 4)))
    error = 1-round(accuracy_score(y_test, y_pred), 4)
    error_rates.append((k, error))

f1_results = pd.DataFrame(f1_scores, columns=['K', 'F1 Score'])
error_results = pd.DataFrame(error_rates, columns=['K', 'Error Rate'])
```

In [38]:
```
f1_results, error_results = kN_best_k(X_train=X_train, X_test=X_test, y_train=y_train, y_test=y_test)
knn_fi_error(f1_results, error_results)
```



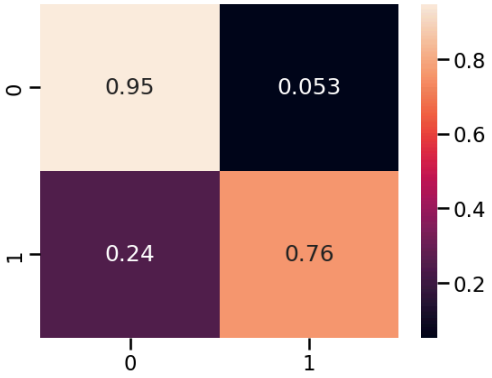
In [39]:
```
knn = KNeighborsClassifier(n_neighbors=31, weights='distance')
knn = knn.fit(X_train, y_train)
knn_y_pred = knn.predict(X_test)
print(classification_report(y_test, knn_y_pred))
print('Accuracy score: ', round(accuracy_score(y_test, knn_y_pred),2))
print('F1 Score', round(f1_score(y_test, knn_y_pred),2))
```

```
              precision    recall  f1-score   support

           0       0.92      0.95      0.93      1552
           1       0.84      0.76      0.80       561

    accuracy                           0.90      2113
   macro avg       0.88      0.85      0.86      2113
weighted avg       0.89      0.90      0.90      2113

Accuracy score:  0.9
F1 Score 0.8
```

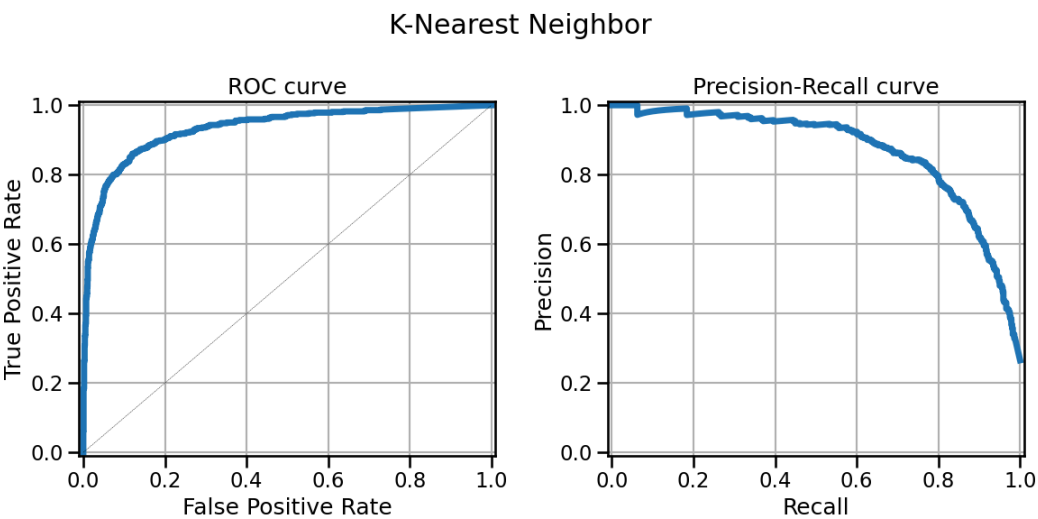
In [40]:
```
knn_metrics = evaluate_metrics(y_test, knn_y_pred)
print(knn_metrics)
```

```
{'accuracy': 0.897, 'recall': 0.897, 'precision': 0.895, 'f1score': 0.895}
```

In [41]:
```
sns.heatmap(confusion_matrix(y_test, knn_y_pred, normalize='true'), annot=True)
plt.show()
```



In [42]:
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=knn, title="K-Nearest Neighbor")
```



Suport Vector Machine (SVM)

In [43]:
```
svm_model = SVC()
svm_model.fit(X_train, y_train.values.ravel())
```

Out[43]:
```
SVC()
```

In [44]:
```
params_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['poly', 'rbf', 'sigmoid'],
}
```

In [45]:
```
grid_search = GridSearchCV(estimator = svm_model, param_grid = params_grid, scoring='f1', cv=5, verbose = 1)
# Search the best parameters with training data
grid_search.fit(X_train, y_train.values.ravel())
best_params = grid_search.best_params_
best_params
```

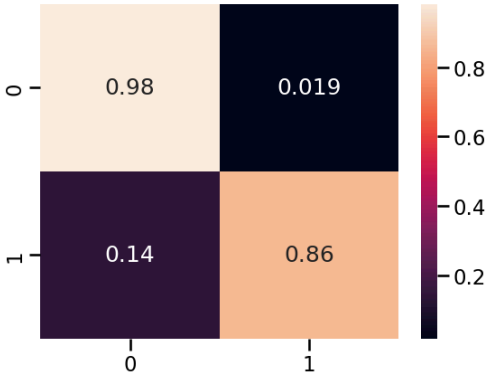
```
Fitting 5 folds for each of 9 candidates, totalling 45 fits
```

Out[45]:
```
{'C': 1, 'kernel': 'rbf'}
```

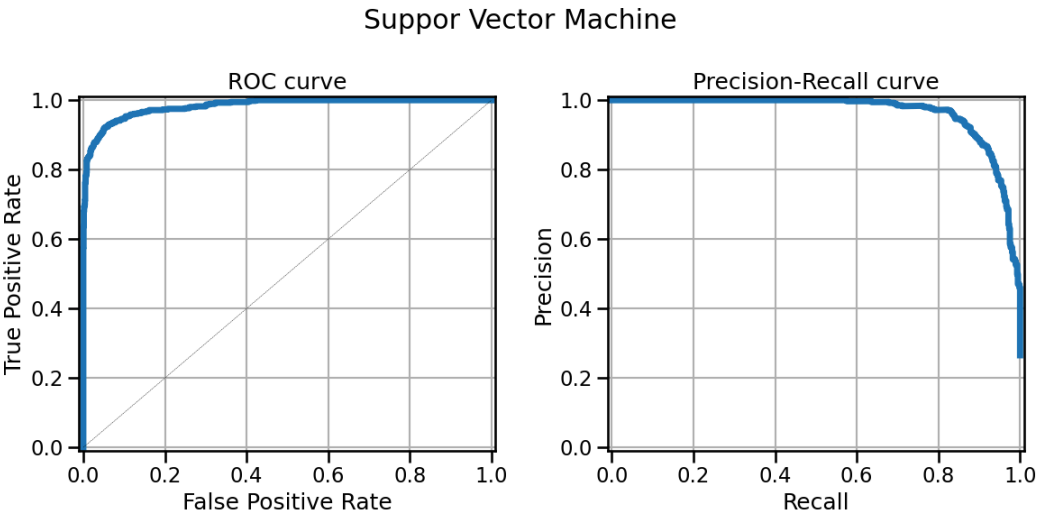
In [46]:
```
svm_model = SVC(C=1, kernel='rbf', probability=True)
svm_model.fit(X_train, y_train.values.ravel())
svm_y_pred = svm_model.predict(X_test)
svm_metrics = evaluate_metrics(y_test, svm_y_pred)
print(svm_metrics)
```

```
{'accuracy': 0.948, 'recall': 0.948, 'precision': 0.948, 'f1score': 0.947}
```

In [47]:
```
sns.heatmap(confusion_matrix(y_test, svm_y_pred, normalize='true'), annot=True)
plt.show()
```



In [48]:
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=svm_model, title="Suppor Vector Machine")
```



Decision Tree

In [49]:
```
params_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [5, 10, 15, 20],
    'min_samples_leaf': [10, 20, 25, 30]
}
```

In [50]:
```
dt_model = DecisionTreeClassifier(random_state=42)
```

In [51]:
```
grid_search = GridSearchCV(estimator = dt_model, param_grid = params_grid, scoring='f1', cv = 5, verbose = 1)
grid_search.fit(X_train, y_train.values.ravel())
best_params = grid_search.best_params_
best_params
```

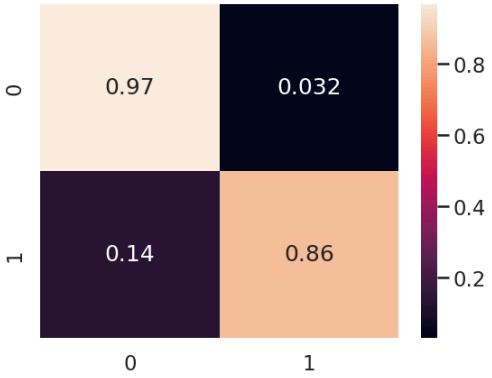
```
Fitting 5 folds for each of 32 candidates, totalling 160 fits
```

Out[51]:
```
{'criterion': 'gini', 'max_depth': 10, 'min_samples_leaf': 20}
```

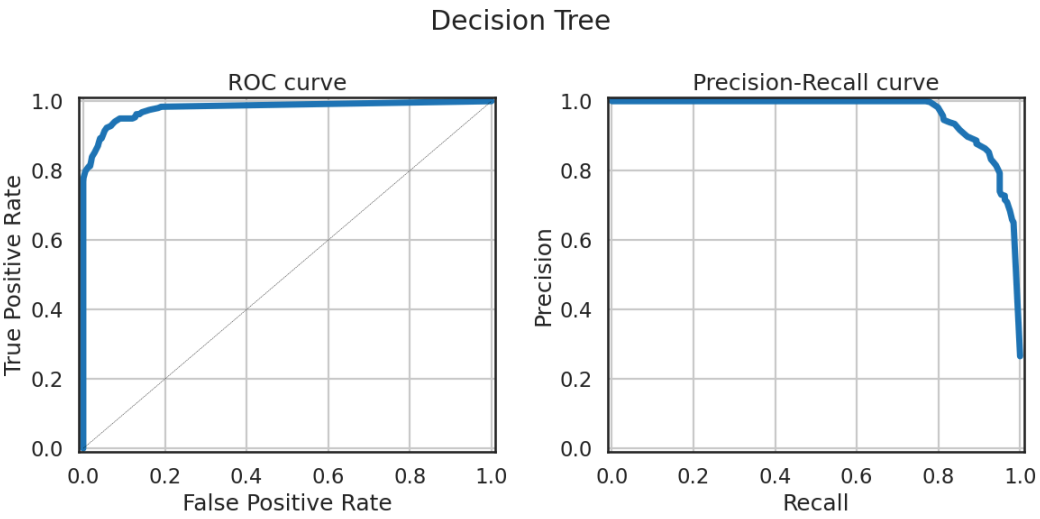
In [52]:
```
dt_model = DecisionTreeClassifier(criterion='gini', max_depth=10, min_samples_leaf=20, random_state=42)
dt_model.fit(X_train, y_train)
dt_y_pred = dt_model.predict(X_test)
dt_metrics = evaluate_metrics(y_test, dt_y_pred)
print(dt_metrics)
```

```
{'accuracy': 0.94, 'recall': 0.94, 'precision': 0.939, 'f1score': 0.939}
```

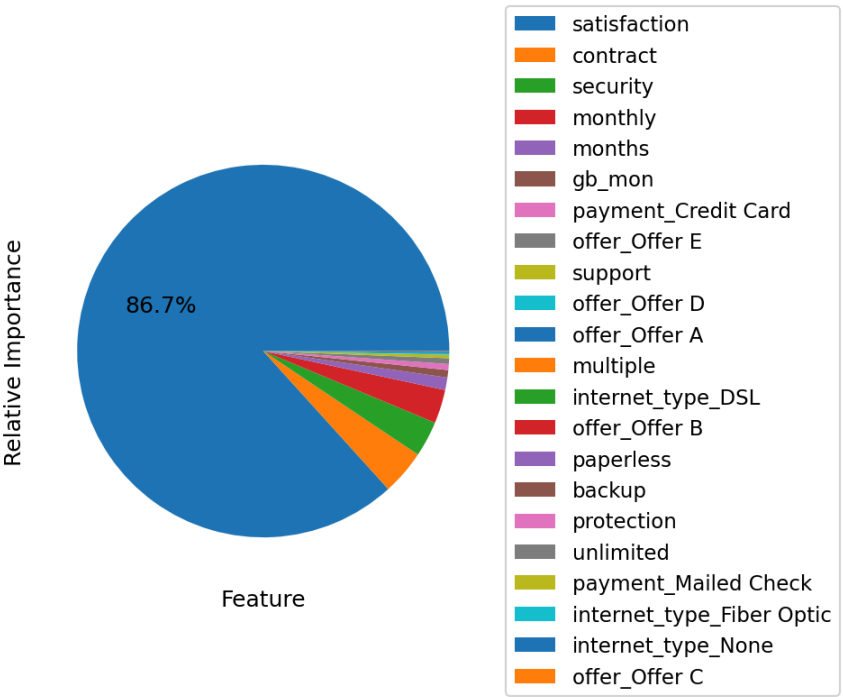
In [339…
```
sns.heatmap(confusion_matrix(y_test, dt_y_pred, normalize='true'), annot=True)
plt.show()
```



In [340…
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=dt_model, title="Decision Tree")
```



In [62]:
```
show_features_importance(dt_model)
```



Random Forest

In [64]:
```
RF = RandomForestClassifier(oob_score=True, random_state=42, warm_start=True, n_jobs=-1)
```

In [65]:
```
oob_list = list()
for n_trees in [15, 20, 30, 40, 50, 100, 150, 200, 300, 400]:
# Use this to set the number of trees
    RF.set_params(n_estimators=n_trees)
    # Fit the model
    RF.fit(X_train, y_train)
    # Get the oob error
    oob_error = 1 - RF.oob_score_
    # Store it
    oob_list.append(pd.Series({'n_trees': n_trees, 'oob': oob_error}))

rf_oob_df = pd.concat(oob_list, axis=1).T.set_index('n_trees')
rf_oob_df
```

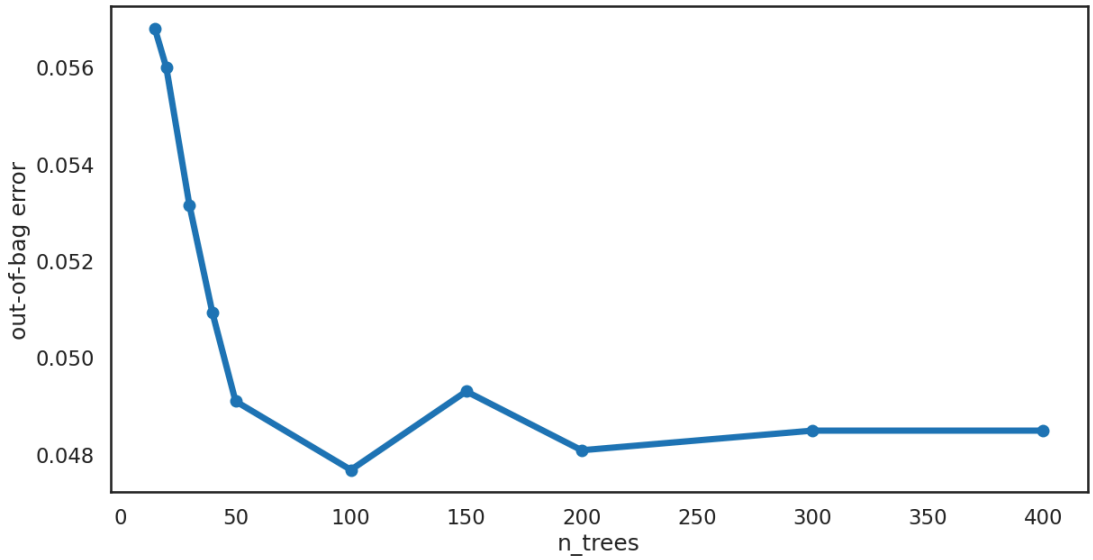
```
/home/melchor/Documentos/VSCode/Coursera/IBM_ML/final_project/.venv/lib/python3.10/site-packages/sklearn/ensemble/_forest.py:612: UserWarning: Some inputs do not have OOB scores. This probably means too few trees were used to compute any reliable OOB estimates.
  warn(
/home/melchor/Documentos/VSCode/Coursera/IBM_ML/final_project/.venv/lib/python3.10/site-packages/sklearn/ensemble/_forest.py:612: UserWarning: Some inputs do not have OOB scores. This probably means too few trees were used to compute any reliable OOB estimates.
  warn(
```

Out[65]:

n_trees	oob
15.0	0.056795
20.0	0.055984
30.0	0.053144
40.0	0.050913
50.0	0.049087
100.0	0.047667
150.0	0.049290
200.0	0.048073
300.0	0.048479
400.0	0.048479

In [66]:
```
sns.set_context('talk')
sns.set_style('white')
ax = rf_oob_df.plot(legend=False, marker='o', figsize=(14, 7), linewidth=5)
ax.set(ylabel='out-of-bag error')
```

Out[66]: [Text(0, 0.5, 'out-of-bag error')]



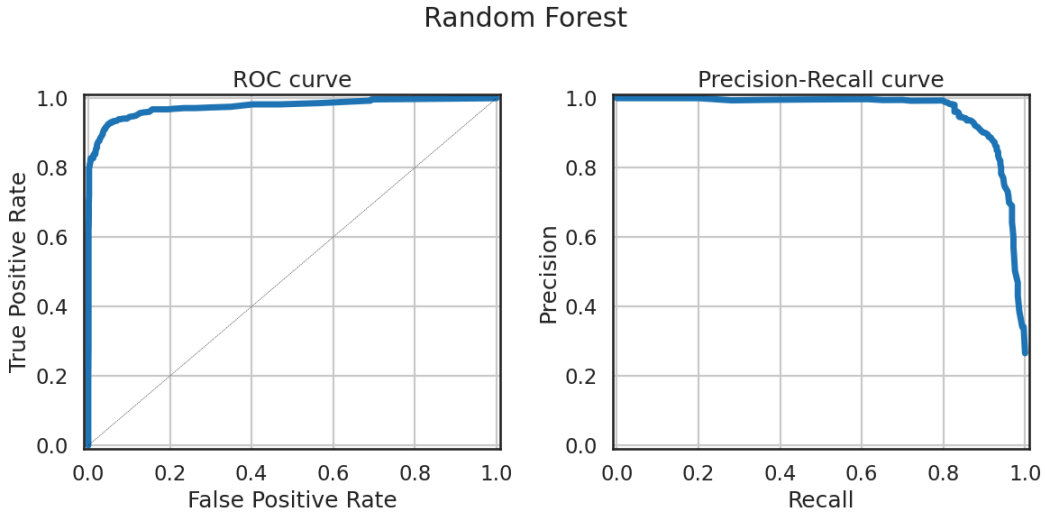
In [67]:
```
RF = RandomForestClassifier(oob_score=True, random_state=42, warm_start=True, n_jobs=-1)
RF.set_params(n_estimators=100)
```

Out[67]:
```
RandomForestClassifier(n_jobs=-1, oob_score=True, random_state=42,
                       warm_start=True)
```

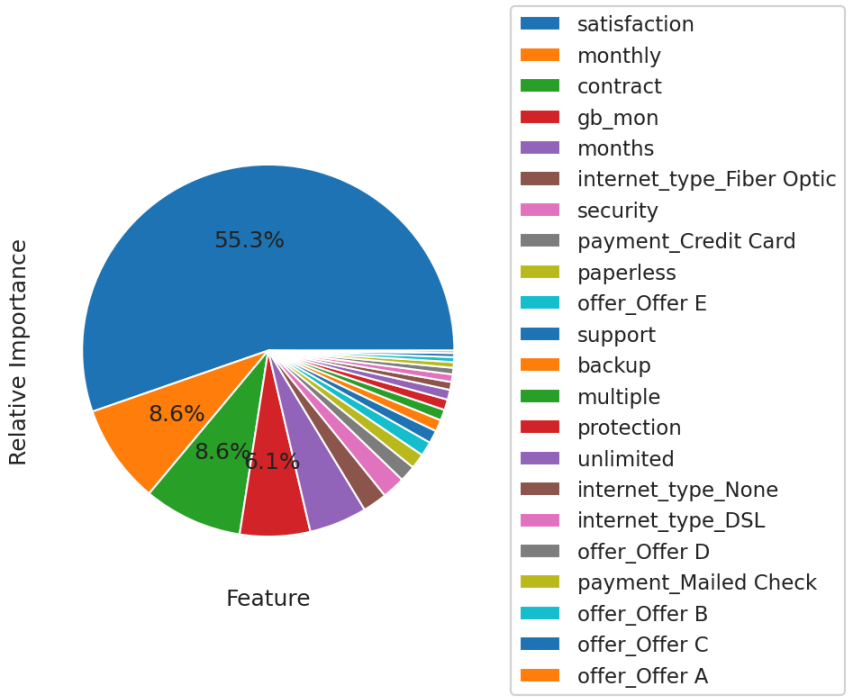
In [68]:
```
RF.fit(X_train, y_train)
rf_y_pred = RF.predict(X_test)
rf_metrics = evaluate_metrics(y_test, rf_y_pred)
print(rf_metrics)
```

```
{'accuracy': 0.946, 'recall': 0.946, 'precision': 0.946, 'f1score': 0.945}
```

In [69]:
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=RF, title="Random Forest")
```



In [70]:
```
show_features_importance(RF)
```



ExtraTreesClassifier

In [80]:
```
EF = ExtraTreesClassifier(oob_score=True, random_state=42, warm_start=True, bootstrap=True, n_jobs=-1)
```

In [81]:
```
oob_list = list()
for n_trees in [15, 20, 30, 40, 50, 100, 150, 200, 300, 400]:
    # Use this to set the number of trees
    EF.set_params(n_estimators=n_trees)
    EF.fit(X_train, y_train)
    # oob error
    oob_error = 1 - EF.oob_score_
    oob_list.append(pd.Series({'n_trees': n_trees, 'oob': oob_error}))

et_oob_df = pd.concat(oob_list, axis=1).T.set_index('n_trees')
et_oob_df
```

```
/home/melchor/Documentos/VSCode/Coursera/IBM_ML/final_project/.venv/lib/python3.10/site-packages/sklearn/ensemble/_forest.py:612: UserWarning: Some inputs do not have OOB scores. This probably means too few trees were used to compute any reliable OOB estimates.
  warn(
/home/melchor/Documentos/VSCode/Coursera/IBM_ML/final_project/.venv/lib/python3.10/site-packages/sklearn/ensemble/_forest.py:612: UserWarning: Some inputs do not have OOB scores. This probably means too few trees were used to compute any reliable OOB estimates.
  warn(
```

Out[81]:

n_trees	oob
15.0	0.066126
20.0	0.065517
30.0	0.060041
40.0	0.056592
50.0	0.054970
100.0	0.050710
150.0	0.050913
200.0	0.049899
300.0	0.050507
400.0	0.049696

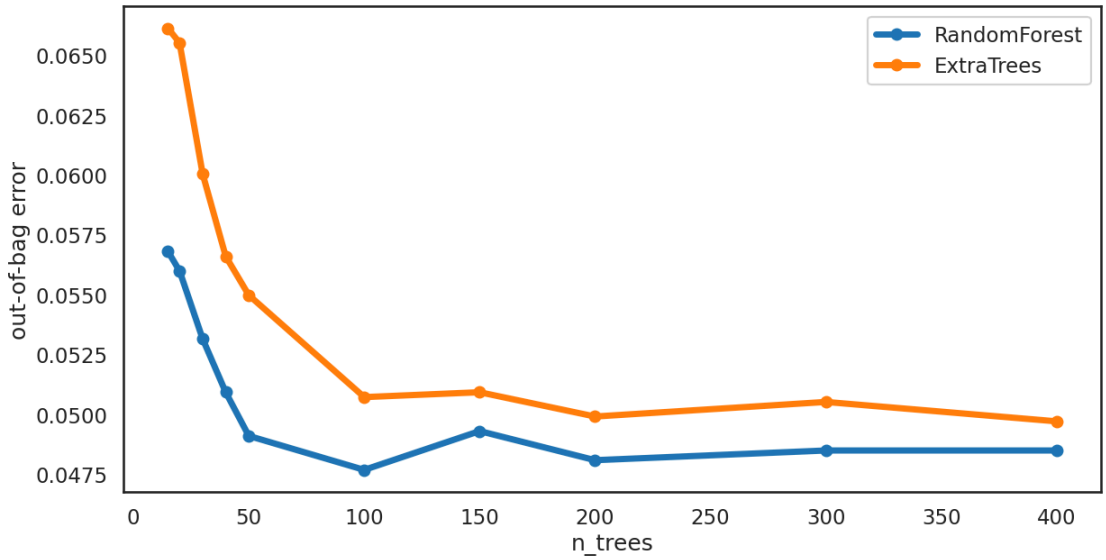
In [82]:
```
oob_df = pd.concat([rf_oob_df.rename(columns={'oob':'RandomForest'}), et_oob_df.rename(columns={'oob':'ExtraTrees'})], axis=1)
oob_df
```

Out[82]:

n_trees	RandomForest	ExtraTrees
15.0	0.056795	0.066126
20.0	0.055984	0.065517
30.0	0.053144	0.060041
40.0	0.050913	0.056592
50.0	0.049087	0.054970
100.0	0.047667	0.050710
150.0	0.049290	0.050913
200.0	0.048073	0.049899
300.0	0.048479	0.050507
400.0	0.048479	0.049696

In [83]:
```
sns.set_context('talk')
sns.set_style('white')
ax = oob_df.plot(marker='o', figsize=(14, 7), linewidth=5)
ax.set(ylabel='out-of-bag error')
```

Out[83]:
```
[Text(0, 0.5, 'out-of-bag error')]
```



In [84]:
```
EF = RandomForestClassifier(oob_score=True, random_state=42, warm_start=True, n_jobs=-1)
EF.set_params(n_estimators=100)
```

Out[84]:
```
RandomForestClassifier(n_jobs=-1, oob_score=True, random_state=42,
                       warm_start=True)
```

In [85]:
```
EF.fit(X_train, y_train)
```

Out[85]:
```
RandomForestClassifier(n_jobs=-1, oob_score=True, random_state=42,
                       warm_start=True)
```

In [88]:
```
EF_y_pred = RF.predict(X_test)
```

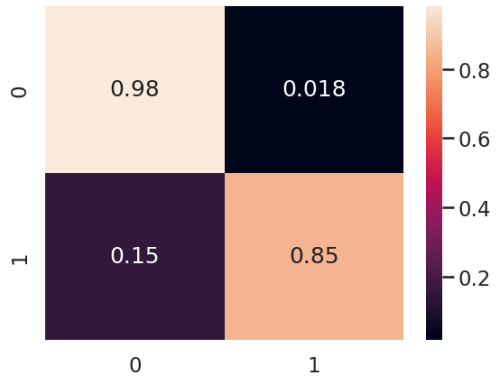
In [89]:
```
EF_cr = classification_report(y_test, EF_y_pred)
print(EF_cr)
```

```
              precision    recall  f1-score   support

           0       0.95      0.98      0.96      1552
           1       0.94      0.85      0.89       561

    accuracy                           0.95      2113
   macro avg       0.95      0.91      0.93      2113
weighted avg       0.95      0.95      0.95      2113
```

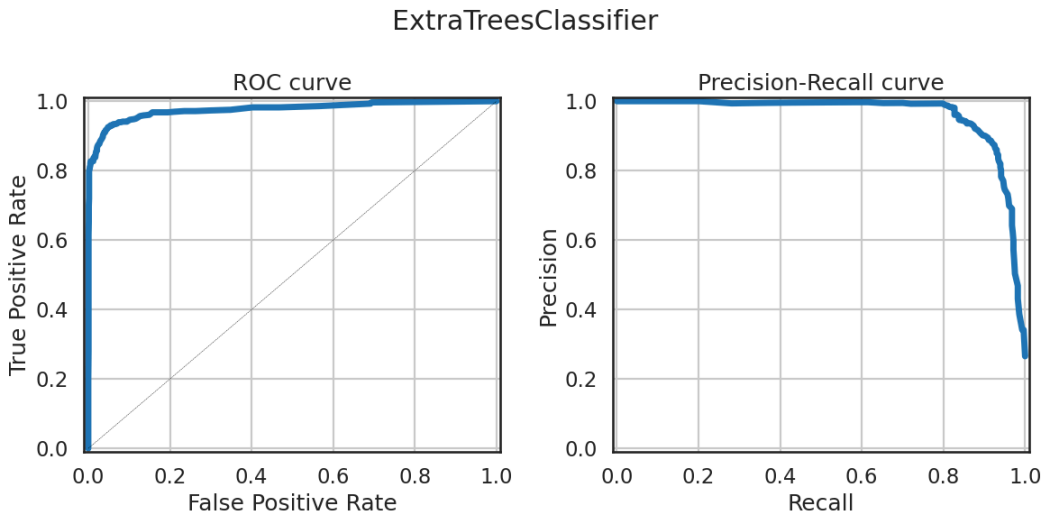
In [90]:
```
EF_metrics = evaluate_metrics(y_test, EF_y_pred)
print(EF_metrics)
```

```
{'accuracy': 0.946, 'recall': 0.946, 'precision': 0.946, 'f1score': 0.945}
```

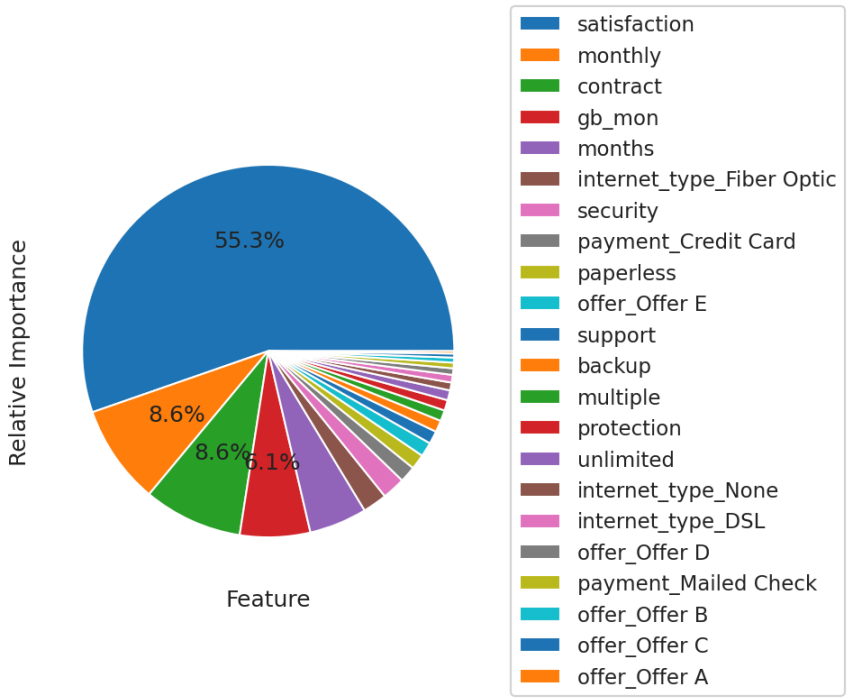
In [91]:
```
sns.heatmap(confusion_matrix(y_test, EF_y_pred, normalize='true'), annot=True)
plt.show()
```



In [92]:
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=EF, title="ExtraTreesClassifier")
```



In [93]:
```
show_features_importance(EF)
```



Boosting and Stacking

In [94]:
```
error_list = list()
# Iterate through various possibilities for number of trees
tree_list = [15, 25, 50, 75, 100, 150, 200]
for n_trees in tree_list:
    # Initialize the gradient boost classifier
    GBC = GradientBoostingClassifier(n_estimators=n_trees, random_state=42)
    # Fit the model
    print(f'Fitting model with {n_trees} trees')
    GBC.fit(X_train.values, y_train.values)
    y_pred = GBC.predict(X_test)
    # Get the error
    error = 1.0 - accuracy_score(y_test, y_pred)
    # Store it
    error_list.append(pd.Series({'n_trees': n_trees, 'error': error}))

error_df = pd.concat(error_list, axis=1).T.set_index('n_trees')
error_df
```

```
Fitting model with 15 trees
Fitting model with 25 trees
Fitting model with 50 trees
Fitting model with 75 trees
Fitting model with 100 trees
Fitting model with 150 trees
Fitting model with 200 trees
```

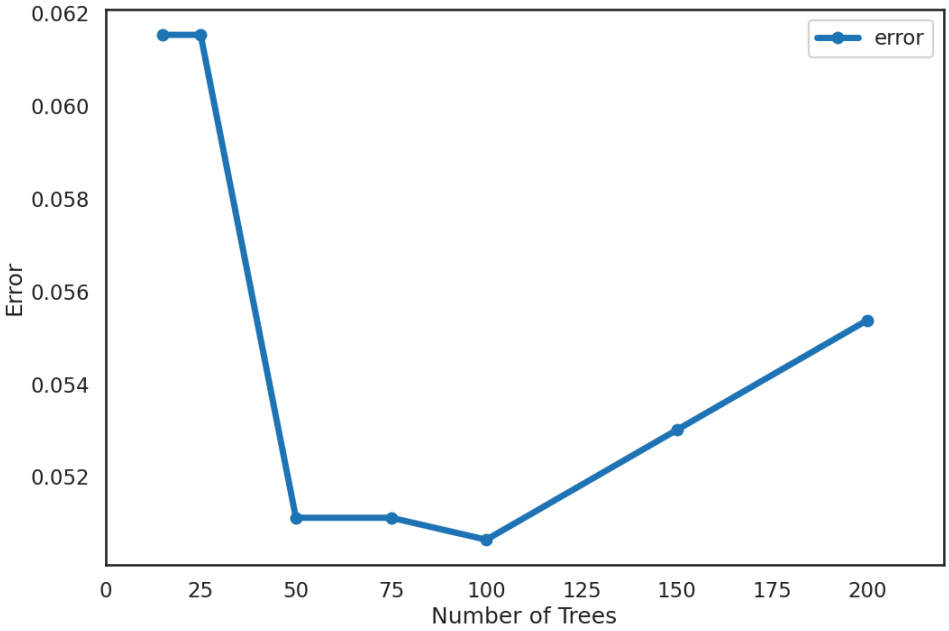
Out[94]:

n_trees	error
15.0	0.061524
25.0	0.061524
50.0	0.051112
75.0	0.051112
100.0	0.050639
150.0	0.053005
200.0	0.055372

In [95]:
```
sns.set_context('talk')
sns.set_style('white')
# Create the plot
ax = error_df.plot(marker='o', figsize=(12, 8), linewidth=5)
# Set parameters
```

```
ax.set(xlabel='Number of Trees', ylabel='Error')
ax.set_xlim(0, max(error_df.index)*1.1)
```

Out[95]: (0.0, 220.00000000000003)



In [96]:
```
param_grid = {'n_estimators': tree_list,
    'learning_rate': [0.1, 0.01, 0.001, 0.0001],
    'subsample': [1.0, 0.5],
    'max_features': [1, 2, 3, 4]
}
```

In [97]:
```
GV_GBC = GridSearchCV(GradientBoostingClassifier(random_state=42), param_grid=param_grid, scoring='accuracy', n_jobs=-1)
```

In [98]:
```
GV_GBC = GV_GBC.fit(X_train, y_train)
```

In [99]:
```
GV_GBC.best_estimator_
```

Out[99]:
```
GradientBoostingClassifier(max_features=3, random_state=42, subsample=0.5)
```

In [100…
```
BS_y_pred = GV_GBC.predict(X_test)
```

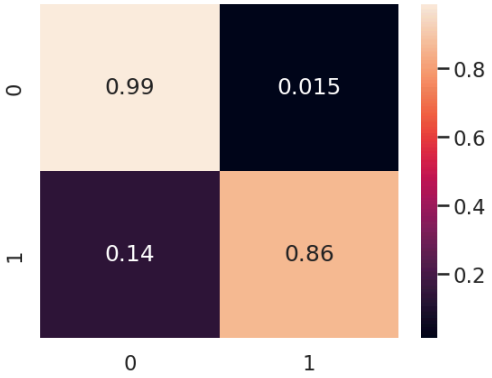
In [101…
```
print(classification_report(y_test, BS_y_pred))
```

```
              precision    recall  f1-score   support

           0       0.95      0.99      0.97      1552
           1       0.95      0.86      0.90       561

    accuracy                           0.95      2113
   macro avg       0.95      0.92      0.94      2113
weighted avg       0.95      0.95      0.95      2113
```

In [102…
```
BS_metrics = evaluate_metrics(y_test, BS_y_pred)
print(BS_metrics)
```

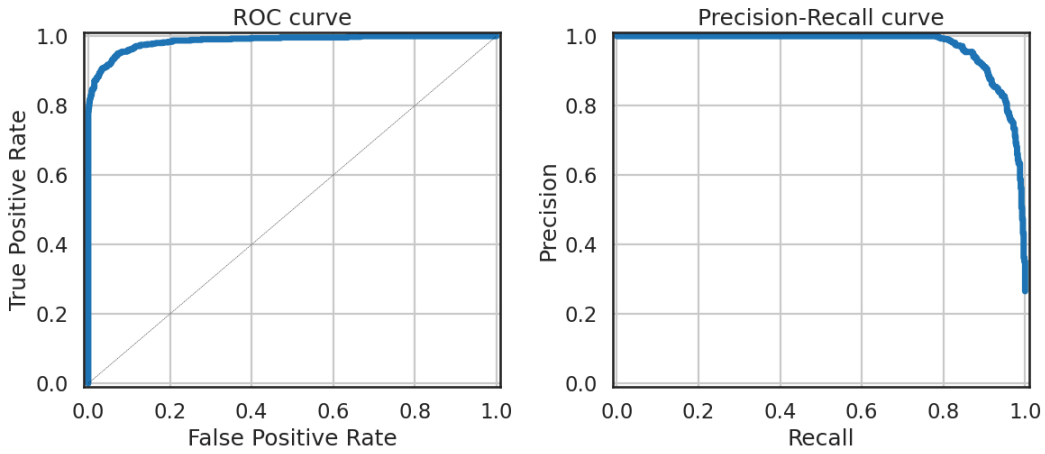
```
{'accuracy': 0.952, 'recall': 0.952, 'precision': 0.952, 'f1score': 0.951}
```

In [103…
```
sns.heatmap(confusion_matrix(y_test, BS_y_pred, normalize='true'), annot=True)
plt.show()
```



In [104…
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=GV_GBC, title="Boosting and Stacking")
```

Boosting and Stacking



AdaBoost Classifier

In [106…
```
ABC = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1))
```

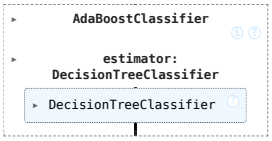
In [107…
```
param_grid = {'n_estimators': [50, 100, 150, 200], 'learning_rate': [0.01, 0.001]}
```

In [108…
```
GV_ABC = GridSearchCV(ABC, param_grid=param_grid, scoring='accuracy', n_jobs=-1)
```

In [109…
```
GV_ABC = GV_ABC.fit(X_train, y_train)
```

In [110…
```
GV_ABC.best_estimator_
```

Out[110…



In [111…
```
ABC_y_pred = GV_ABC.predict(X_test)
```

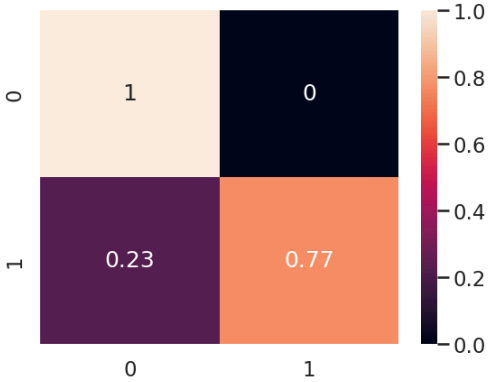
In [112…
```
print(classification_report(ABC_y_pred, y_test))
```

```
              precision    recall  f1-score   support

           0       1.00      0.92      0.96      1682
           1       0.77      1.00      0.87       431

    accuracy                           0.94      2113
   macro avg       0.88      0.96      0.91      2113
weighted avg       0.95      0.94      0.94      2113
```

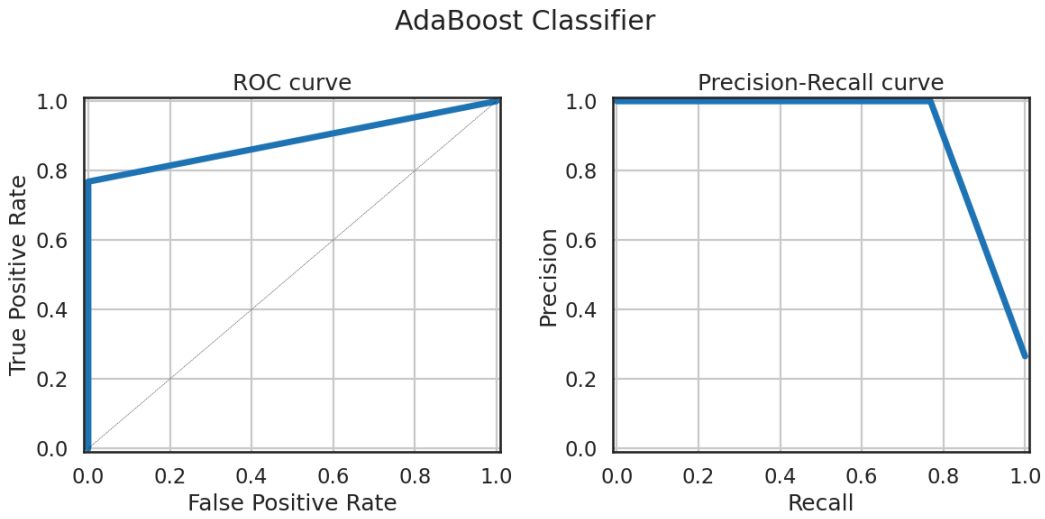
In [113…
```
ABC_metrics = evaluate_metrics(y_test, ABC_y_pred)
print(ABC_metrics)
```

```
{'accuracy': 0.938, 'recall': 0.938, 'precision': 0.943, 'f1score': 0.936}
```

In [114…
```
sns.heatmap(confusion_matrix(y_test, ABC_y_pred, normalize='true'), annot=True)
plt.show()
```



In [115…
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=GV_ABC, title="AdaBoost Classifier")
```



Stacked

In [116…
```
LR_L2 = LogisticRegressionCV(Cs=10, cv=4, penalty='l2',solver='liblinear').fit(X_train, y_train)
```

In [117…
```
LR_L2_y_pred = LR_L2.predict(X_test)
print(classification_report(LR_L2_y_pred, y_test))
```

```
              precision    recall  f1-score   support

           0       0.98      0.95      0.97      1599
           1       0.87      0.95      0.91       514

    accuracy                           0.95      2113
   macro avg       0.93      0.95      0.94      2113
weighted avg       0.96      0.95      0.95      2113
```

In [118…
```
evaluate_metrics(y_test, LR_L2_y_pred)
```

Out[118…
```
{'accuracy': 0.954, 'recall': 0.954, 'precision': 0.954, 'f1score': 0.953}
```

In [119…
```
estimators = [('LR_L2', LR_L2), ('GBC', GV_GBC)]
```

In [120…
```
VC_lr_gbc = VotingClassifier(estimators, voting='soft')
VC_lr_gbc = VC_lr_gbc.fit(X_train, y_train)
```

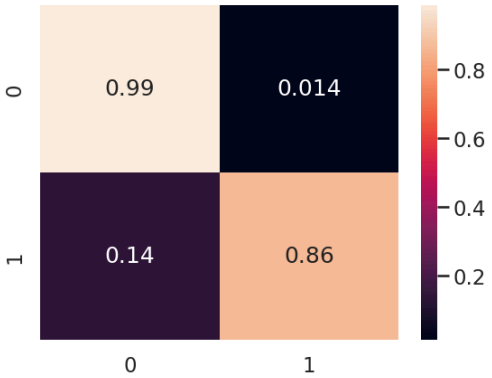
In [121…
```
VC_lr_gbc_y_pred = VC_lr_gbc.predict(X_test)
print(classification_report(y_test, VC_lr_gbc_y_pred))
```

```
              precision    recall  f1-score   support

           0       0.95      0.99      0.97      1552
           1       0.96      0.86      0.91       561

    accuracy                           0.95      2113
   macro avg       0.96      0.92      0.94      2113
weighted avg       0.95      0.95      0.95      2113
```

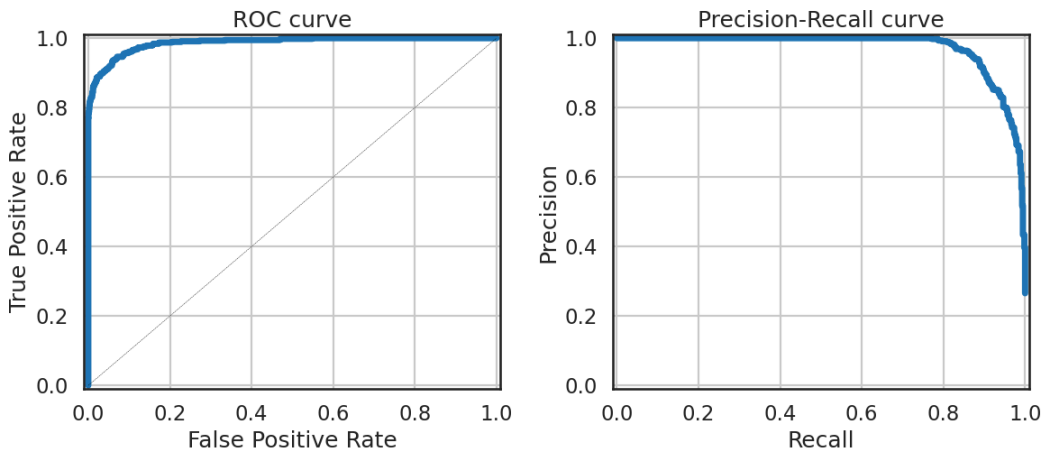
In [122…
```
stk_metrics = evaluate_metrics(y_test, VC_lr_gbc_y_pred)
print(stk_metrics)
```

```
{'accuracy': 0.954, 'recall': 0.954, 'precision': 0.954, 'f1score': 0.953}
```

In [123…
```
sns.heatmap(confusion_matrix(y_test, VC_lr_gbc_y_pred, normalize='true'), annot=True)
plt.show()
```



In [124…
```
show_ROC_PressRecCurv(X_test=X_test, y_test=y_test, model=VC_lr_gbc, title="Stacked Model")
```



In [125…
```
models_metrics = lr_metrics
```

In [126…
```
models_metrics = pd.concat([models_metrics, pd.Series(knn_metrics,name="k-nn"),
                            pd.Series(svm_metrics,name="Support Vector"),
                            pd.Series(dt_metrics,name="Desicion Tree"),
                            pd.Series(rf_metrics,name="Reandom Forest"),
                            pd.Series(EF_metrics,name="ExtraTreesClassifier"),
                            pd.Series(BS_metrics,name="Boosting/Stacking"),
                            pd.Series(ABC_metrics,name="AdaBoost"),
                            pd.Series(stk_metrics,name="Stacked")], axis=1)
models_metrics
```

Out[126…

	lr	l1	l2	k-nn	Support Vector	Desicion Tree	Reandom Forest	ExtraTreesClassifier	Boosting/Stacking	AdaBoost	Stacked
precision	0.950684	0.952993	0.954013	0.895	0.948	0.939	0.946	0.946	0.952	0.943	0.954
recall	0.950308	0.953147	0.954094	0.897	0.948	0.940	0.946	0.946	0.952	0.938	0.954
f1score	0.949289	0.952503	0.953433	0.895	0.947	0.939	0.945	0.945	0.951	0.936	0.953
accuracy	0.950308	0.953147	0.954094	0.897	0.948	0.940	0.946	0.946	0.952	0.938	0.954
auc	0.917799	0.927130	0.927775	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

In [127…
```
models_metrics.rename(columns={"lr":"LogisticRegression", "l1":"LogisticRegression_L1", "l2":"LogisticRegression_L2"}, inplace=True)
```

In [128…
```
models_metrics.T.sort_values("f1score", ascending=False)
```

Out[128…

	precision	recall	f1score	accuracy	auc
LogisticRegression_L2	0.954013	0.954094	0.953433	0.954094	0.927775
Stacked	0.954000	0.954000	0.953000	0.954000	NaN
LogisticRegression_L1	0.952993	0.953147	0.952503	0.953147	0.927130
Boosting/Stacking	0.952000	0.952000	0.951000	0.952000	NaN
LogisticRegression	0.950684	0.950308	0.949289	0.950308	0.917799
Support Vector	0.948000	0.948000	0.947000	0.948000	NaN
Reandom Forest	0.946000	0.946000	0.945000	0.946000	NaN
ExtraTreesClassifier	0.946000	0.946000	0.945000	0.946000	NaN
Desicion Tree	0.939000	0.940000	0.939000	0.940000	NaN
AdaBoost	0.943000	0.938000	0.936000	0.938000	NaN
k-nn	0.895000	0.897000	0.895000	0.897000	NaN

In [ ]: